Deep Learning based Bidiretional LSTM network for Comment Toxicity Classification

Dyuti Oindrela, Sumaiya Morshed

Bsc in Computer Science, Asian University for Women, Chattogram 4000, Bangladesh
Email: dyuti.oindrela@auw.edu.bd, sumaiya.morshed@auw.edu.bd

***Abstract*—With the advent of social media usage, toxic comments have been increasing at an alarming rate. This toxicity requires to be kept in check while trying to eradicate it to make digital platforms safer with only healthy community engagements. With this goal in mind, several recent studies have been carried out for toxic comment classification making it an effective research area. Similarly, this research paper has tried to classify a dataset of comments into toxicity ranges with labels like toxic, severe toxic, threat, insult, identity hate, etc. A deep learning-based Bidirectional LSTM network has been utilized to train the auto-detection of toxic comments and further classification. A user-friendly interface, gradio has been applied to manually fill in input at the test stage and get the desired classified results. This model shows an acuuracy rate of 97%.**

Keywords - Deep Learning, Bi-directional LSTM, Optimization, Toxic Comment Classification, Threat, Epoch, Accuracy

1. Introduction

The unprecedented increase in user-created content like posts, reviews, comments, and other material produced and shared by netizens has been fueled by the meteoric rise and proliferation of social media platforms, as well as the emergence of numerous online forums, communities, and websites which ensures increased user engagement and participation. While the sharing and exchange of user-generated content can promote healthy discourse, open expression of viewpoints, and a sense of community engagement, concurrently it unlocks the possibility of harmful, offensive, and toxic speeches. This hurtful form of expression sometimes ends up undermining positive online interactions by inflicting harm, marginalizing certain groups of people, and impacting individuals and communities.

Researchers such as Huang [1] have categorized and delineated any form of text, remark, post, or any form of online correspondence that degrades, demonstrates hatred or contempt towards a person or group based on their identity like lineage, complexion, ethnic origin, sexual preference, faith or political ideology. Furthermore, threats of violence, harassment, bullying, or derogatory information that undermines that standard of respectful communication can be considered abusive remarks. The domain of virulent verbal attacks encompasses a broad range of detrimental statements that attack peoples' sentiments and dehumanize people based on their inherent characteristics or affiliations.

Multiple digital ecosystems have an unrestricted and minimally regulated environment that allows online media to be shared without appropriate oversight or moderation. Oftentimes, due to the absence of uniform and vigilant monitoring, the online channels witness a surge of abusive comments. According to the BBC, in a worldwide study, it has been found that 1% out of 6 teenagers have been subjected to online bullying. World Health Organization's (WHO) European regional director stated that after COVID-19 as people's attention and activity shifted to cyberspace, online hate speeches increased as well. As social media platforms grapple with the immense challenge of regulating and moderating the proliferation of

hate comments and toxic speech online, they find themselves confronted with a paradoxical situation. The very algorithms designed to promote engagement and drive user activity inadvertently contribute to the spread of hate speech, as such content is often flagged and prioritized by these systems due to its ability to generate high levels of interaction and response. Consequently, this algorithmic bias towards engaging and inflammatory content has led to a concerning boom in the availability and dissemination of hate data across various online platforms. In this research paper, we propose a comprehensive and multi-faceted approach for toxic comment classification that combines and integrates a deep learning-based method utilizing a Bidirectional Long Short-Term Memory (BiLSTM) neural network architecture.

The rest of the paper is arranged as follows: Section 2 includes related work, Section 3 deals with the proposed methodology, and Section 4 and Section 5 contain the result and conclusion respectively.

2. Related Works

With the abundance of toxic language and comments across various social media platforms, researchers from all across the globe have been utilizing different Artificial intelligence models to detect abusive remarks. Some used Machine learning frameworks, some used Deep learning models and some used a combination of both to address the escalating digital pandemic.

Li et al., explored different frameworks for automated toxic comment detection, using both classical Machine Learning (ML) methods as well as deep learning approaches [2]. Initially, they used the Naive Bayes, Support Vector Machine (SVM) classifier as baseline models acquiring an F1 score of 68.33% and an Exact Match (EM) score of 87.57%. Later, to increase the performance of the model, the researchers used two deep learning models such as Long Short-Term Memory (LSTM) networks and Bidirectional Encoder Representations from Transformers (BERT). Leveraging the weighted loss strategy during the training process, they mitigated the problem of imbalanced data which increased their F1 score and EM score to 81.19% and 95.54% respectively.

In a research study conducted by Akhter et al., the researchers carried out the study to detect abusive remarks and language in Urdu and Roman Urdu comments. Five diverse Machine Learning models such as Naive Bayes, Support Vector Machine (SVM), Instance-Based Learning, Logistic Regression, and JRip have been implemented in this particular study. Furthermore, Convolutional Neural Network(CNN), Long Short-Term Memory (LSTM), Bidirectional LSTM and Convolutional LSTM, these four deep learning models have been enforced on a large dataset over 10,000 Roman Urdu comments and a smaller dataset of 2,000 Urdu comments. Finally, the CNN model's accuracy of 96.2% on Urdu comments and accuracy of 91.4% on Roman Urdu comments outweigh the performances of other models [3].

Islam et al., in response to the increase in cyberbullying and abusive messages on social media platforms, developed a way to automatically identify toxic comments in digital media by combining Natural Language Processing (NLP) with ML [4]. They utilized two distinct feature sets such as Bag of Words and Term Frequency-Inverse Document Frequency (TF-IDF), to examine efficiency in terms of accuracy of four ML models such as SVM, Random Forest, Decision Tree, and Naive Bayes. Among all the classifiers, the SVM model showcased the most accuracy.

Abbasi et al., [5] upon discovering a couple of ML algorithms associating certain descriptors like "Muslim", "Black", "Jewish", and "White" as higher toxicity rating indicators, implemented advanced deep learning classifiers for multi-label toxic comments classification. The research follows two approaches. First, the classification of multi-labeled toxic comments based on religious belief, and second, multi-labeled classification based on inherent ethnicity. GloVe, Word2vec, FastText, and regular embedding layers without embedding layers have been used and evaluated in this research. The study findings demonstrate that CNN outperforms the rest of the models involved in the research in terms of classifying toxic remarks in both of the approaches.

M PLAZA-DEL-ARCO et al., [6] developed their hate speech detection model by implementing Sentiment analysis in their multi-task learning

approach. While a lot of the ML models only focus on hate speech detection as the main purpose of their research, this particular group of researchers incorporated polarity and emotion classification to relate to offensive behavior. They use a transformer-based model to detect hate remarks in Spanish tweets. The outcome elucidated that merging emotional and polarity information results in accurate hate speech detection across the dataset in comparison to hate-only baselines.

3. Proposed Approach

In this paper, we applied a deep learning approach to classify the toxicity of comments focusing on a Bidirectional LSTM (long short term memory) network, a powerful architecture sequence modeling with the hypothesis of it bearing effective results in capturing more nuanced evaluation of language in comments.

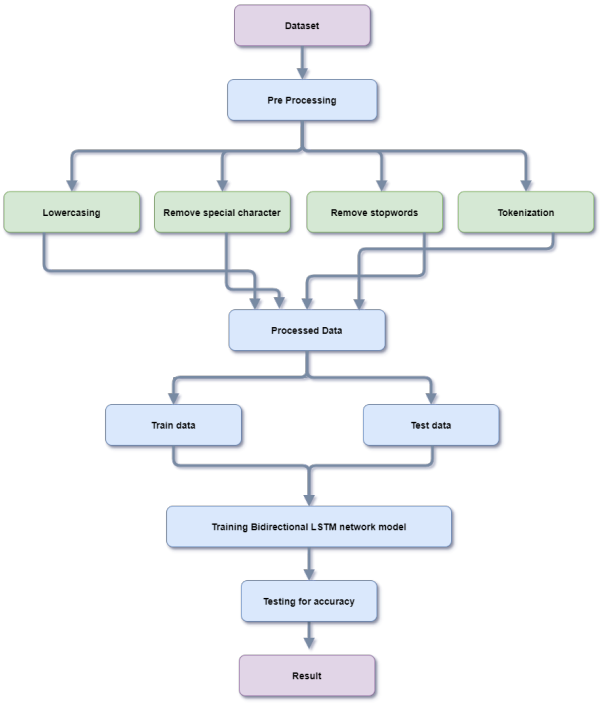

Figure 1. Work Flow

3.1. Dataset

In this research paper, the dataset used in this study has been collected from a Kaggle [7] Challenge hosted by Jigsaw. The name of the dataset is entitled "Jigsaw Toxic Comment Classification Challenge". This dataset contains around 1,59,571 English comments which are categorized into six categories toxic, severe toxic, obscene, threat, insult, and identity threat. The dataset was pre-labeled whether a corresponding comment was a toxic, severe toxic, obscene, threat, insult, identity threat or not using 0 and 1. The entire dataset contains 15,294 toxic comments, 1,595 severe toxic comments, 8,449 obscene comments, 478 threatful comments, 7,877 insulting comments, and 1,405 identity hate comments. It contains special characters, and stop words which are removed from the dataset while pre-processing. In our proposed model, we have only used the six different categories of labels to detect and classify comment toxicity. Figure 2 shows the graphical representation of the number of comments under each of the category.

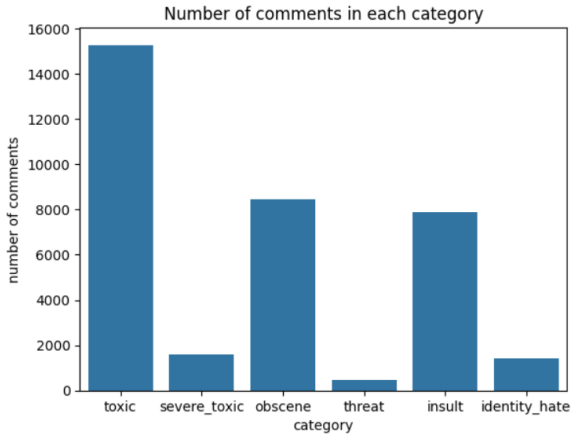

Figure 2. Number of comments in each category

3.2. Pre-Processing

Pre-processing is done on the raw text data to clean it up and make it more consistent. This entails changing the text's font style to lowercase, eliminating non-alphabetic letters, and filtering out stopwords.

- Lowercasing: In many language processing models supposedly being case-sensitive, lowercasing is a common step to convert all characters to their lowercase to ensure consistency. Even though the converted words might still bear the same meaning as before.

- Removing special characters: To focus better on meaningful words, all special characters

along with punctuations are removed except lowercase letters and white spaces.

- Removing stopwords: Frequently occurring words for example “a”, “is”, “the” etc which don’t put much into the meaning of a sentence are often considered stopwords. To reduce noise caused by these stopwords, a list of common English stopwords is imported into the code from nltk.corpus module and an iteration running over through the text data looks for the stopwords, removes them, and joins the remaining words using a space(‘ ’)as a delimiter.

For clear understanding, the before and after of the pre-processing of data is shown in Figure 2 and Figure 3 as a visible comparison.

	id	comment_text	toxic	severe_toxic	obscene	threat	insult	identity_hate
0	0000997932d777bf	Explanation\nWhy the edits made under my usern...	0	0	0	0	0	0
1	000103f0d9cfb60f	D'aww! He matches this background colour I'm s...	0	0	0	0	0	0
2	000113f07ec002fd	Hey man, I'm really not trying to edit war. It...	0	0	0	0	0	0
3	0001b41b1c6bb37e	"\nMore\nI can't make any real suggestions on ...	0	0	0	0	0	0
4	0001d958c54c6e35	You, sir, are my hero. Any chance you remember...	0	0	0	0	0	0
5	00025465d4725e87	"\n\nCongratulations from me as well, use the ...	0	0	0	0	0	0
6	0002bcb3da6cb337	COCKSUCKER BEFORE YOU PISS AROUND ON MY WORK	1	1	1	0	1	0
7	00031b1e95af7921	Your vandalism to the Matt Shirvington article...	0	0	0	0	0	0
8	00037261f536c51d	Sorry if the word 'nonsense' was offensive to ...	0	0	0	0	0	0
9	00040093b2687caa	alignment on this subject and which are contra...	0	0	0	0	0	0

Figure 3. Dataset before pre-processing

	id	comment_text	toxic	severe_toxic	obscene	threat	insult	identity_hate
0	0000997932d777bf	explanation edits made username hardcore metal...	0	0	0	0	0	0
1	000103f0d9cfb60f	daww matches background colour im seemingly st...	0	0	0	0	0	0
2	000113f07ec002fd	hey man im really trying edit war guy constant...	0	0	0	0	0	0
3	0001b41b1c6bb37e	cant make real suggestions improvement wondere...	0	0	0	0	0	0
4	0001d958c54c6e35	sir hero chance remember page thats	0	0	0	0	0	0
5	00025465d4725e87	congratulations well use tools well talk	0	0	0	0	0	0
6	0002bcb3da6cb337	cocksucker piss around work	1	1	1	0	1	0
7	00031b1e95af7921	vandalism matt shirvington article reverted pl...	0	0	0	0	0	0
8	00037261f536c51d	sorry word nonsense offensive anyway im intend...	0	0	0	0	0	0
9	00040093b2687caa	alignment subject contrary dulithgow	0	0	0	0	0	0

Figure 4. Dataset after pre-processing

The Pre-processed data is then fed into the tokenizer for further processing and then into the subsequent model stags for our text classification.

- *Tokenizer:* In this stage, the text data is transformed into a numerical representation better suited for neural networks. The created tokenizer object considers the most frequent words in the training data and assigns a unique identifier to each of them. Unidentified words are assigned a special token.

- *Padding:* Padding ensures that the sequences of varying lengths after tokenization are made into inputs of the same size for the neural network. Post padding is used here meaning adding zeros at the end of each sequence to make them of the same length.

- *Train-Test split:* Our dataset is now split into a train set and a test of ratio 8:2 for further processing and training the algorithm accordingly.

3.3. Model Architecture

This section further describes the architecture of the model that has been used for toxicity classification.

(1) *Embedding layer:* This layer takes the integer ID representing a word from the tokenizer’s vocabulary and turns it into a vector representation to map into a dense vector of a predefined size of 128 in this case, acting as a bridge between the neural network numeral processing capability and the pre-processed text data. A higher dimensionality (128) allows for more effective capturing of more complex and semantic relationships between the words. A higher dimensionality also increases the number of parameters to learn.

(2) *First Bi-directional LSTM Layer:* Within the Bidirectional LSTM layer lies the core of the model architecture for our desired classification. Bidirectional LSTM (long short-term memory) layers excel in sequence modeling in text classification by capturing dependencies between words. Bidirectional LSTM is different than our traditional LSTM model for its two-directional (forward LSTM and backward LSTM) structure. A bidirectional LSTM network is

used when we want to be able to utilize both past and future input features for a specific time in the sequence tagging task [8]. Backpropagation through time (BPTT) is used to train bidirectional LSTM networks [9].

- Forward LSTM follows natural reading order (left to right) for sequence processing. Considering the current word and its hidden state from the previous step, the internal cell and the hidden state of the current state are updated all while capturing information on the processed words till that stage.

- Backward LSTM then processes in reverse sequence (right to left) capturing information about future words in the sequence. After processing of the sequence is done both forward LSTM and backward LSTM generate their hidden states which are combined to form an enriched representation capturing information from both directions.

(3) *Dropout Layers:* 50% of the activation is randomly dropped by this layer from the previous layer. More dropout layers help to improve model generalization and to prevent overfitting. And the final one helps in regularization. This segment is further explained in Section 3.4

(4) *Second Bi-directional LSTM Layer:* Similar to the first layer, this layer has two LSTMs (64 hidden units each) which process the sequence in both directions. The output is set to have a single vector representation of the entire comment's context

(6) *Dense Layers (128, ReLU):* With a ReLU activation function, dense layers transform the high-dimensional vector from the LSTM into a lower-dimensional space (128 units).

(6) *Third Dropout Layer:* It is the final dropout layer for regularization.

(7) *Output Layer (6, Sigmoid):* This layer has 6 units and uses a sigmoid activation. The sigmoid function outputs values between 0 and 1, by representing the probability of a comment belonging to each category.

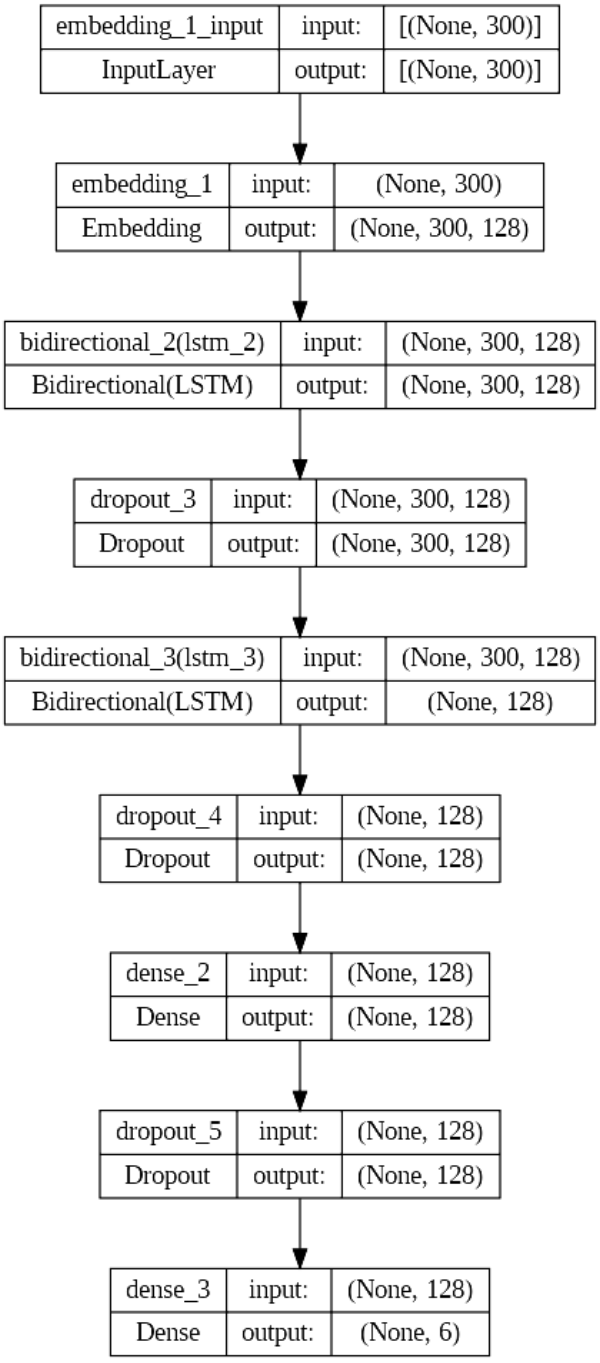

Figure 5. Model Summary

- *ReLU Activation function:* The equation for the ReLU (Rectified Linear Unit)activation function is $f(x) = max(0, x)$. This means that if the input, x is greater than or equal to zero, the function returns the input, but if the input is negative, the function returns zero.

The output of the ReLU function can range from zero to infinity.

- *Sigmoid Activation function:* The activation function sigmoid $f(x) = \sigma(x) = 1/1 + e^{-}z$ is used to introduce non-linearity in the model. Here z is the value from the previous output layer in the neural network and x is the input for input layer.

Figure 6 and Figure 7 demonstrates the activation graphs for both the activation functions.

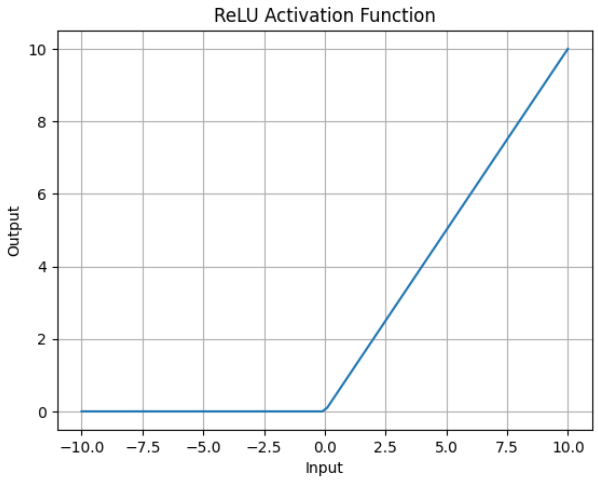

Figure 6. Rectified Linear Unit (ReLU) activation Graph

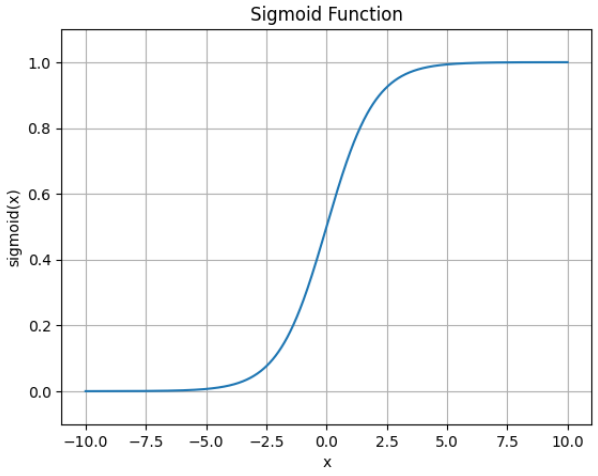

Figure 7. Sigmoid Function Activation Graph

3.4. Hyper parameter tuning

A few hyperparameters that have a further effect on the proposed model are mentioned in this section. It is important to mention that the following list is not comprehensive and there may be more unclear hyperparameters in effect. Hyperparameter tuning (e.g. methods like grid search random search) is a crucial step in determining the ideal settings of the following hyperparameters for better results.

- *MAX_FEATURES* (200000)*:* The maximum number of words that will be considered by the tokenizer is defined in this code segment. If the value is increased, the model will be allowed to handle a greater range of vocabulary. On the contrary, it can increase the training time and usage of memory.

- *MAX_LENGTH* (300)*:* The maximum length of the sequence for the comments is defined as 300. Any shorter comment than this will be padded with the special token, and any longer comment will be shortened. Due to this hyperparameter data consistency format for the model is ensured. Considering this length to be any lower than this might cut off important text segments. Whereas any lengthier limit might cause a waste of memory usage through unnecessary padding computation.

- *Embedding dimension:* The embedding dimension gives control over the feature complexity the model may learn from every word. Dimensions higher than the allotted number might facilitate the capturing of more enriched semantic relationships but it also causes to involvement of more training data expanding the usage of computational resources.

- *Number of LSTM Units* (64)*:* The number of hidden units in each LSTM layer is regulated as a hyperparameter. These units then appraise the value of the model capacity in learning the complex patterns in data sequencing. Again, increasing the number of units may improve the model capacity but also may lead

to overfitting due to handling errors.

- *Dropouts*(0.5)*:* A common regularization technique, dropout improves the model's ability to generalize unseen data. A very high rate of dropout might obstruct the model from effective learning whereas a lower rate might prove to be inefficient as a regularizer hence tuning of this hyperparameter is very crucial.

- *Epochs:* This hyperparameter is used to determine the number of times the whole training dataset is made to pass through the model while controlling the learning rate from the training data. Less epochs might make the model underfitting, whereas too many epochs may lead to an overfitted model.

- *Optimizer (Adam):* The Adam optimizer algorithm preserves the gradient's direction by adjusting the learning rate to each input weight vector rather than to each weight [10]. The training speed of the mode can be greatly affected by the choice of the optimizer. In various deep-learning approaches like this one, Adam is a popular choice due to its effectiveness.

- *Loss Function (Binary Cross-Entropy):* The binary cross entropy function measures the difference between the model's accuracy and predictions. Then it adjusts the model and its weights so that the difference is minimized. In addition to being useful for similarity searches, binary descriptors can also be used as discriminant features in classification [11] It is also used to determine how the model learns from its mistakes

- *Batch size* (32)*:* The number of data points processed by the model at each training step is determined by the batch size. Batch size manages memory usage by affecting the training speed. A smaller batch may result in slower speed but for complex models, it might provide with might provide with better convergence.

4. Result

The accuracy for the built model is 97%. This result shows the potential of the Bidirectional LSTM model through its effectiveness in the classification of toxic comments traversing through the intricate relationship between each word in the given dataset. To find the result the model has been compiled with the Adam optimizer, Binary cross entropy as loss, and accuracy as metrics. Figure 8 shows the loss vs validation loss and accuracy vs validation accuracy for the training data set with the epoch increment while the epoch was set for 10.

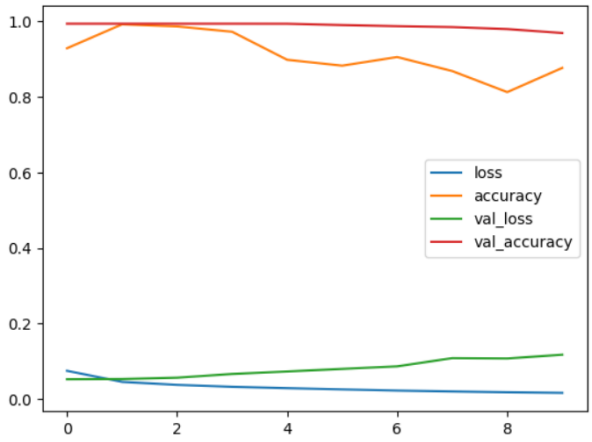

Figure 8. loss vs val_loss, accuracy vs val_accuracy

to validate the results and further user-level testing, Gradio (a user-friendly interface) was used to take manual input and show the output text as classifications of the text if toxic("You piece of * Go die in a ditch.") see Figure 10. in this output segment the comment was marked positive for the classes 'toxic', 'severe toxic', 'identity threat', 'insult' and 'obscene' and negative for 'threat' as the comment doesn't bear any threatful remarks. And for the not-toxic class comment ("I love your Pretty face") the interface generates all negative answers supporting the claim of this paper

5. Conclusion

The expansion of social media has resulted in an increase in poisonous remarks and hate speech online. This toxicity is a huge danger to the creation of secure and healthy digital communities. Addressing this issue is critical for fostering good engagements

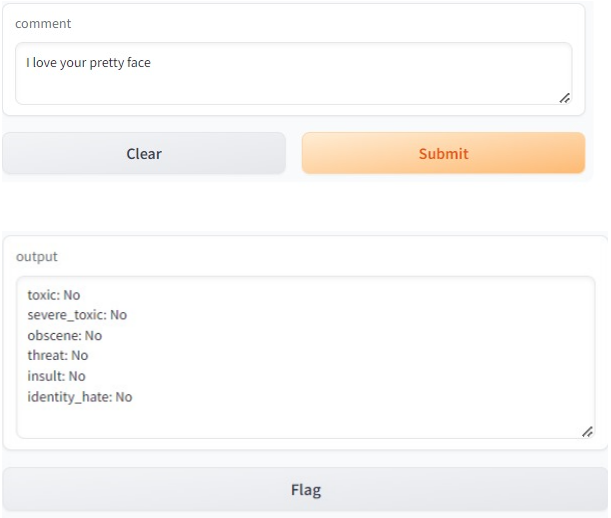

Figure 9. Output sample 1

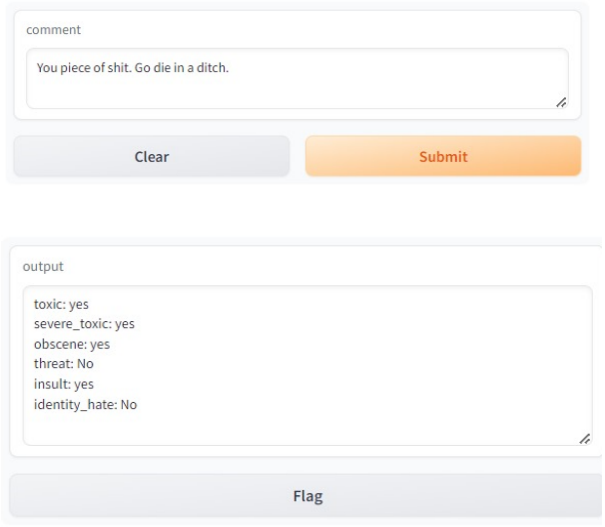

Figure 10. Output sample 2

and creating inclusive online environments.This study aimed to provide an efficient system for automatically categorizing comments into varied levels of toxicity, including categories like toxic, severe toxic, threat, insult, and identity hatred. The suggested approach excelled in detecting and categorizing harmful comments by leveraging the capabilities of deep learning, namely a Bidirectional LSTM network.

After intensive training on a broad dataset, the model reached an amazing 97% accuracy rate, demonstrating its potential for real-world applications. Furthermore, the use of a user-friendly interface, Gradio, enables seamless manual entry and retrieval of categorized findings during testing.While this study indicates a huge step forward, the fight against online toxicity is ongoing. Future work should focus on improving the model's accuracy, broadening its ability to handle multi modal data, and assuring its ethical and responsible deployment.

Finally, successful mitigation of toxic remarks and hate speech online necessitates a multimodal strategy that includes technology developments, community participation, and governmental reforms. By integrating cutting-edge AI solutions like the one provided in this study with larger social initiatives, we can help to create more inclusive, courteous, and powerful digital places for everyone.

For the completion of the Deep learning project both of the teammates (Oindrela & Morshed) equally participated in the coding segment, presentation segment, and report writing segment which helped which the successful execution of the entire project.

6. Future Scope

A potential future path is to use pre-trained language models like BERT, RoBERTa, and XLNet to classify harmful comments via transfer learning. These models, which have been pre-trained on enormous datasets, are capable of capturing extensive semantic and contextual information, potentially enhancing performance. Fine-tuning them on task-specific labeled data may result in increased accuracy in spotting poisonous, severe toxic, threatening, crude, and identity-attacking remarks. However, difficulties like as inherent biases, computational requirements, and responsible deployment must be addressed with thorough analysis and mitigation techniques.

References

[1] Z. Huang, W. Xu, and K. Yu, "Bidirectional lstm-crf models for sequence tagging," *arXiv preprint arXiv:1508.01991*, 2015.

[2] P. Tare, "Toxic comment detection and classification," in *31st Conference on Neural Information Processing Systems (NIPS 2017)*, 2017, pp. 1–6.

[3] M. P. Akhter, Z. Jiangbin, I. R. Naqvi, M. AbdelMajeed, and T. Zia, "Abusive language detection from social media comments using conventional machine learning and deep learning approaches," *Multimedia Systems*, vol. 28, no. 6, pp. 1925–1940, Dec. 2022.

[4] M. M. Islam, M. A. Uddin, L. Islam, A. Akter, S. Sharmin, and U. K. Acharjee, "Cyberbullying detection on social networks using machine learning approaches," in *2020 IEEE Asia-Pacific Conference on Computer Science and Data Engineering (CSDE)*, 2020, pp. 1–6.

[5] A. Abbasi, A. R. Javed, F. Iqbal, N. Kryvinska, and Z. Jalil, "Deep learning for religious and continent-based toxic content detection and classification," *Scientific Reports*, vol. 12, no. 1, p. 17478, 2022.

[6] F. M. Plaza-Del-Arco, M. D. Molina-González, L. A. Ureña-López, and M. T. Martín-Valdivia, "A multi-task learning approach to hate speech detection leveraging sentiment analysis," *IEEE Access*, vol. 9, pp. 112 478–112 489, 2021.

[7] C. adams, J. E. Jeffrey Sorensen, L. Dixona, M. McDonald, and W. C. nithum, "Toxic comment classification challenge," 2017. [Online]. Available: https://kaggle.com/competitions/jigsaw-toxic-comment-classification-challenge

[8] A. Graves and J. Schmidhuber, "Framewise phoneme classification with bidirectional lstm and other neural network architectures," *Neural networks*, vol. 18, no. 5-6, pp. 602–610, 2005.

[9] M. Bodén, "A guide to recurrent neural networks and backpropagation," 12 2001.

[10] Z. Zhang, "Improved adam optimizer for deep neural networks," in *2018 IEEE/ACM 26th International Symposium on Quality of Service (IWQoS)*, 2018, pp. 1–2.

[11] L. Liu and H. Qi, "Learning effective binary descriptors via cross entropy," in *2017 IEEE Winter Conference on Applications of Computer Vision (WACV)*, 2017, pp. 1251–1258.